

Code-Layout, Readability and Reusability

Date	29/09/2026
Team ID	UNM1425LegalEase: AI-Powered Legal Document Generator
Project Name	LegalEase: AI-Powered Legal Document Generator
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent Python-based formatting is documented across the project.
2	Proper File Structure	Files and folders are logically organized	Yes	main.py, routes.py, ai_core, formatting utilities, and Streamlit UI are separated.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names such as document_type, parties, terms, and dates describe their inputs.
4	Function / Method Names	Functions are descriptively named	Yes	generate_document, sanitize_text, format_docx, format_pdf, and format_html_preview are descriptive.
5	Code Comments	Inline and block comments explain logic	Partial	The source documents the workflow, but extensive inline code comments are not shown.
6	Modular Design	Code is split into reusable functions/modules	Yes	API, AI generation, formatting, and UI responsibilities are separated into modules/functions.
7	No Redundant Code	Duplicate or unused code is removed	Partial	The architecture is modular, but the source PDF does not show enough code to verify all redundancy removal.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Validation/testing steps are documented, but detailed exception-handling code is not shown.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	main.py	Python / FastAPI	Backend setup and root endpoint	High

2	routes.py	Python / FastAPI / Pydantic	POST /generate document-generation route	High
3	GeminiDocumentGenerator	Python / Google Gemini API	AI generation called by backend route	High
4	format_docx, format_pdf, format_html_preview	Python / python-docx / FPDF / HTML	Document export and HTML preview	High
5	Streamlit UI components	Python / Streamlit	Input, preview, editing, and downloads	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	The documented architecture separates backend, routes, AI core, formatting, and UI.
Readability	4	Descriptive names and a clearly organized workflow are shown in the project source.
Reusability	5	Generation and formatting functions are separated for reuse across document outputs.
Documentation / Comments	3	Architecture and workflow are documented; extensive inline comments are not shown in the source.
Overall Score	4	Overall structure is modular and reusable; source evidence for comments and detailed exception handling is limited.